Natural-Language Mathematical Statements to Formal Statements

Autoformalization as alignment, not translation

DSA5210 – Session 4

Ma Jiajun – May 22, 2026

Part I. Opening Case Study

One symbol, many formal objects

One Symbol, Many Formal Objects: Integral

On the board only this:

$$\int_a^b f(x)\,dx$$

> Question: to write this as a Lean theorem statement, what is missing?

The missing piece is not Lean syntax. It is the mathematical object.

Possible readings

1. Riemann integral on a compact real interval.
2. Lebesgue integral wrt Lebesgue measure.
3. Bochner integral of a Banach-space-valued function.
4. Oriented interval integral $\int_a^b$.
5. Set integral over $[a, b]$, $(a, b]$, or another measurable set.
6. Variable upper limit: $x \mapsto \int_a^x f(t)\,dt$.
7. Antiderivative relation: $F' = f$.

Running Example: Improper Riemann Exists, Lebesgue Does Not

$$\int_1^\infty \frac{\sin x}{x}\,dx \text{ exists.}$$

Reading A – improper Riemann integral: **true**

```
∃ L : ℝ,
  Tendsto
    (fun R : ℝ ⇒ ∫ x in (1:ℝ)..R, Real.sin x / x)
    atTop
    (𝓝 L)
```

Finite interval integrals have a limit as $R \to \infty$.

Reading B – Lebesgue integrable on $[1, \infty)$: **false**

```
¬ IntegrableOn
    (fun x : ℝ ⇒ Real.sin x / x)
    (Set.Ici 1)
```

$\int_1^\infty |\sin x/x|\,dx = \infty$. Lebesgue integrability needs absolute integrability; this is only conditional convergence.

Putnam 2025 B2: Geometry Becomes Integral Ratios

Let $f : [0, 1] \to [0, \infty)$ be strictly increasing and continuous. Let R be the region under $y = f(x)$ on $[0, 1]$. Let x_1 be the x-coordinate of the centroid of R. Let x_2 be the x-coordinate of the centroid of the solid generated by rotating R about the x-axis. Prove $x_1 < x_2$.

```
theorem putnam_2025_b2
    (f : ℝ → ℝ)
    (hf_cont : ContinuousOn f (Icc 0 1))
    (hf_mono : StrictMonoOn f (Icc 0 1))
    (hf_nonneg : ∀ x ∈ Icc (0 : ℝ) 1, 0 ≤ f x) :
    (∫ x in (0:ℝ)..1, x * f x) /
      (∫ x in (0:ℝ)..1, f x) <
    (∫ x in (0:ℝ)..1, x * (f x) ^ 2) /
      (∫ x in (0:ℝ)..1, (f x) ^ 2) := by
  sorry
```

The formal statement does not define R, centroid, or solid of revolution. It uses the analytic formulas directly.

Good formalization may replace a geometric narrative by an analytically equivalent statement that the library can handle. The π in the volume formula cancels in the quotient; denominator positivity is a proof obligation, not an extra hypothesis.

Gemini Draft: Correct Shape, Lean Syntax Fixes

Two layers on this slide

1. Reuse Lean syntax from earlier sessions to repair the draft.
2. After repair, see that Gemini's **mathematical shape** is right.

Common fixes: - module path / `import Mathlib`; - open `Set Real MeasureTheory Interval`; - `noncomputable` section; - `π` as `Real.pi`; - parameterize over `f`.

```
noncomputable section

def area_R (f : ℝ → ℝ) : ℝ :=
  ∫ x in (0:ℝ)..1, f x

def x1 (f : ℝ → ℝ) : ℝ :=
  (∫ x in (0:ℝ)..1, x * f x) / area_R f

def vol_solid (f : ℝ → ℝ) : ℝ :=
  Real.pi * ∫ x in (0:ℝ)..1, (f x)^2

def x2 (f : ℝ → ℝ) : ℝ :=
  (Real.pi * ∫ x in (0:ℝ)..1, x * (f x)^2) /
    vol_solid f
```

AxiomMath inlines the analytic formulas; Gemini's definition-layer version names `area`, `x1`, `vol_solid`, `x2`. Both express the same analytic comparison. Presentation choice, not error.

"Indefinite Integral" Is Not A Defined Object

(A) Derivative side

```
-- predicate: F' = f x at x
HasDerivAt : (𝕜 → F) → F → 𝕜 → Prop

-- total function: junk value 0 if no derivative
deriv : (𝕜 → F) → 𝕜 → F
```

Module:
`Mathlib.Analysis.Calculus.Deriv.Basic`.
`HasDerivAt` is a `Prop`. `deriv` is a value.

(B) Definite integral side

```
-- oriented interval integral, returns a value
intervalIntegral
  : (ℝ → E) → ℝ → ℝ → Measure ℝ → E
-- notation: ∫ x in a..b, f x ∂μ
```

Module:
`Mathlib.MeasureTheory.Integral.IntervalIntegral.Ba`
Built on Bochner `MeasureTheory.integral`.

There is **no** `indefiniteIntegral`, `antiderivative`, or `primitive` def in mathlib. Every informal use of "indefinite integral" must reduce to (A) or (B). FTC is the theorem connecting them, **not** a definition.

Three Informal Phrases, Two Primitives

Natural language	Real identity	Mathlib expression
"F is an antiderivative of f"	predicate on side (A)	`∀ x, HasDerivAt F (f x) x`
"the antiderivatives of f"	set carved out by that predicate	`{F | ∀ x, HasDerivAt F (f x) x}`
"$\int_a^x f(t)\,dt$"	a function on side (B)	`fun x ⇒ ∫ t in a..x, f t`

The first row uses **only** the derivative side. There is no integral in it.

The second row is a **set-builder**, not a `def`. mathlib has no `antiderivatives` definition.

The third row uses **only** the integral side. It is just a lambda over `intervalIntegral`.

Every alignment of "the indefinite integral of f" has to commit to one of three forms above – and therefore to one of the two primitives (A) or (B).

`deriv f x = 0` Is **Not** Evidence Of Differentiability

Core warning. `deriv f x = 0` does **not** imply that f is differentiable at x. The equation is **not** evidence of differentiability.

Why

`deriv` is a **total function**: Lean requires a value at every point.

If f is not differentiable at x, then `deriv f x` returns `0` by convention (junk value).

Two interpretations of `deriv f x = 0`

1. f *is* differentiable at x, derivative happens to be 0; **or**
2. f is *not* differentiable at x, `deriv` returned the default 0.

You cannot tell which from the equation alone.

Correct shape for "$f'(0) = 0$": use `HasDerivAt f 0 0` (packs existence + value together), or hold `DifferentiableAt ℝ f 0` as a separate hypothesis before reading off `deriv`. This is exactly why side (A) provides **both** `HasDerivAt` (assertion) and `deriv` (value).

Heaviside Step: The Junk-Value In Action

Setup

$$H(x) = \mathbf{1}_{[0,\infty)}(x)$$

```
def H : ℝ → ℝ :=
  Set.indicator (Set.Ici 0) (fun _ ⇒ 1)
```

H is **not continuous** at 0, so certainly not differentiable there.

Yet in Lean

```
example : deriv H 0 = 0 := by sorry
```

This holds – but via the **junk-value** path, not because the derivative equals 0.

`deriv H 0 = 0` ✓ (true)
`DifferentiableAt ℝ H 0` ✗ (false)
`HasDerivAt H 0 0` ✗ (false)

Reading `deriv f x = 0` as "the derivative at x is 0" is wrong whenever the function is not differentiable there. The compiler accepts the equation; the mathematical content has silently dropped out.

Fundamental Theorem Of Calculus As Statement Shapes

FTC-1

If $F(x) = \int_a^x f(t)\,dt$, then $F'(x) = f(x)$.

```
HasDerivAt
  (fun x ⇒ ∫ t in a..x, f t) (f x) x
```

FTC-2

If $F'(x) = f(x)$, then $\int_a^b f(x)\,dx = F(b) - F(a)$.

```
(∫ x in a..b, f' x) = F b - F a
```

Mathlib theorem names:
`intervalIntegral.integral_eq_sub_of_hasDerivAt`
`intervalIntegral.integral_deriv_eq_sub'`

The real content lives in the statement: - `HasDerivAt`, `HasDerivWithinAt`, or `deriv`? - Where does the derivative hold? - Integrability hypothesis? - Oriented interval? - Codomain structure?

"The fundamental theorem of calculus" $\Longrightarrow$ a family of theorem schemas.

Worked Example: $f' \equiv 0 \Longrightarrow f$ Constant (1/2)

Theorem (informal). If f is continuous on $[a, b]$ and $f' = 0$ on (a, b), then f is constant on $[a, b]$.

The "obvious" formal shape

```
theorem constant_of_deriv_zero
    {a b : ℝ} (hab : a < b) {f : ℝ → ℝ}
    (hcont : ContinuousOn f (Icc a b))
    (hdiff : DifferentiableOn ℝ f (Ioo a b))
    (hderiv : ∀ x ∈ Ioo a b, deriv f x = 0) :
    ∀ x ∈ Icc a b, ∀ y ∈ Icc a b, f x = f y :=
  sorry
```

Reads off the informal sentence directly: continuity on $[a, b]$, differentiability on (a, b), $f' = 0$ on (a, b), conclude constancy.

No Mathlib lemma has this exact shape

Closest closed-interval lemmas in `Mathlib.Analysis.Calculus.MeanValue`:

- `constant_of_derivWithin_zero`
- `constant_of_has_deriv_right_zero`
- `is_const_of_deriv_eq_zero` (no interval)

> None accept "`DifferentiableOn` + `deriv = 0` on `Ioo a b`" as written. The alignment gap is in the **derivative notion**: at endpoints Mathlib uses `derivWithin` / `HasDerivWithinAt`, not `deriv` / `HasDerivAt`.

Worked Example: $f' \equiv 0 \Longrightarrow f$ Constant (2/2)

Real Mathlib API

```
-- closed interval ⟹ derivWithin, not deriv
example
    {a b : ℝ} {f : ℝ → ℝ}
    (hdiff : DifferentiableOn ℝ f (Icc a b))
    (hderiv : ∀ x ∈ Ico a b,
      derivWithin f (Icc a b) x = 0) :
    ∀ x ∈ Icc a b, f x = f a :=
  constant_of_derivWithin_zero
    hdiff hderiv
```

The lemma is `constant_of_derivWithin_zero`. Same lesson as Slide 43 (LeanArchitect / Multivariate Taylor): at endpoints, use the `Within` variants.

Four alignment shifts vs. draft

- `DifferentiableOn` on `Icc` (continuity inherited)
- `derivWithin`, not `deriv`
- condition on `Ico a b`, not `Ioo a b`
- conclusion pinned to $f(a)$, not symmetric

The "deriv = 0 ⟹ constant" theorem has at least two real Mathlib shapes – both use the `Within` variants. **Neither** uses `deriv` or `HasDerivAt` directly.

Mathlib Does Not Start From Textbook Notation

Notation used by the Putnam B2 statement earlier:

```
∫ x in a..b, f x
```

comes from `MeasureTheory.Integral.IntervalIntegral`. Not Riemann sums as primary definition; an oriented interval integral built on the measure-theoretic integral.

```
∫ x in a..b, f x ∂μ
  = ∫ x in Ioc a b, f x ∂μ
  - ∫ x in Ioc b a, f x ∂μ
```

Orientation is part of the API:

```
∫ x in a..a, f x ∂μ = 0
∫ x in b..a, f x ∂μ
  = - ∫ x in a..b, f x ∂μ
∫ x in a..b, f x ∂μ
  + ∫ x in b..c, f x ∂μ
  = ∫ x in a..c, f x ∂μ
```

Formal libraries often choose definitions for theorem engineering, not for matching first-semester textbook wording.

The Underlying Integral Is Bochner Integral

`MeasureTheory.integral` is the Bochner integral.

```
MeasureTheory.integral
  input:
    measure μ on domain α
    function f : α → E
    structure on E
      sufficient for integration
  output:
    element of E
```

Not only real-valued functions; takes values in suitable normed vector spaces.

Common notations:

```
∫ x, f x ∂μ
∫ x in s, f x ∂μ
∫ x in a..b, f x ∂μ
```

> Behind ordinary-looking notation sits a hierarchy:

```
measurable space / measure
  → integrable functions
  → Bochner integral
  → set integral
  → interval integral
```

Formalization step one: locate the concept in the library.

Mathlib Also Has Riemann-Style Integrals

Riemann-style integral lives in a separate module, `BoxIntegral`:

```
BoxIntegral.HasIntegral
BoxIntegral.Integrable
BoxIntegral.integral
BoxIntegral.IntegrationParams.Riemann
```

The same framework hosts Henstock–Kurzweil and McShane via different `IntegrationParams`.

Textbook phrase → Mathlib object:

```
"the Riemann integral"
  → BoxIntegral with
     IntegrationParams.Riemann

"the integral from a to b"
  → intervalIntegral built
     from MeasureTheory.integral
```

Bridge theorems connect box-style and measure-theoretic integrals under suitable hypotheses. The library keeps multiple related definitions; one is the more common proof interface.

What The Integral Case Teaches About Alignment

Template for the rest of the lecture:

```
informal phrase
  "the integral of f from a to b"

formal choices
  type of f
  codomain structure
  measure
  interval convention
  integrability condition
  endpoint behavior
  intended theorem shape
```

> *An informal mathematical statement is often a compressed reference to a whole formal design space.*

The integral example was concrete. Part II abstracts it into a general checklist.

Part II. General Mechanism

From informal sentence to formal statement

The Statement Is The Object

Session 4 builds:

> **a formal theorem statement**
> (not a proof)

Two checks on a formal statement:

1. It typechecks in the formal language.
2. It expresses the intended mathematical meaning of the informal statement.

> Dangerous autoformalization failures pass (1) but fail (2).

Example:

> Informal: *Every subgroup of a cyclic group is cyclic.*

A type-correct but wrong formalization could:

- state the ambient group is cyclic in the conclusion instead of the subgroup;
- use the wrong formal definition of "cyclic".

Alignment Checklist

1. **Objects.** Variables, functions, sets, structures, parameters.
2. **Types.** Each object's type.
3. **Structures.** Algebraic, order, topological, metric, measure, category.
4. **Quantifiers.** Universal, existential, order.
5. **Assumptions.** Explicit vs convention-hidden.
6. **Definitions.** Which library definition corresponds.
7. **Notation.** Overloaded symbols disambiguated.
8. **Statement strength.** Too weak, too strong, just right.
9. **Edge cases.** 0, empty set, equal endpoints, boundaries.
10. **Back-translation.** Translate the formal back; does it still read as the original theorem?

This is the human version of statement validation. Modern systems (Part IV) automate parts of it.

Typechecking Is Not Semantic Alignment

A Lean statement can be well-typed and still be the wrong theorem.

Common failure modes

- **Too weak.** Only a special case.
- **Too strong.** Extra hypotheses, or sharper conclusion.
- **Wrong object.** Nearby but different definition.
- **Wrong scope.** Quantifier order swapped.
- **Wrong ambient structure.** Specialized to $\mathbb{R}$ instead of generic.
- **Wrong convention.** Index from 0 vs 1; open vs closed interval; left derivative vs derivative.

> Formal statement checking is not enough. We also need statement alignment.

Part III. Worked Examples

Simple to research-level alignment problems

The First n Odd Numbers Sum To n^2

Informal:

> The sum of the first n odd numbers is n^2.

Hidden choices

- n is a natural number?
- "first n" means n terms, or up to n?
- Odd numbers as $2i + 1$ or $2i - 1$?
- Index $0..n - 1$ or $1..n$?
- Sum over `Finset.range`, list, recursion?
- Equality in `Nat`, `Int`, `Real`?

A possible Lean statement:

```
example (n : Nat) :
    (∑ i in Finset.range n, (2 * i + 1))
      = n ^ 2 := by
  ...
```

This statement has already made these choices:

- `n : Nat`;
- zero-based indexing;
- first n terms are $1, 3, \ldots, 2n - 1$;
- finite sum over `Finset.range`;
- equality in `Nat`.

Set Distributivity

Informal:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

Hidden choices

- Universe type α?
- A, B, C are `Set α`?
- Extensional set equality?
- $\cap, \cup$ as set ops or lattice ops?

```
variable {α : Type*} (A B C : Set α)

example :
    A ∩ (B ∪ C)
      = (A ∩ B) ∪ (A ∩ C) := by
  ...
```

Even this simple example forces: - a universe `α`; - objects typed as `Set α`; - set lattice notation; - extensional equality.

Good first-pass group exercise: alignment without heavy analysis.

The Inverse Of A Product

Informal:

> The inverse of a product is the product of the inverses in reverse order.

Hidden choices

- Group, monoid-with-inverses, division ring, or category?
- Multiplication commutative or not?
- Conclusion $(ab)^{-1} = b^{-1}a^{-1}$?
- In a commutative setting, reverse order is hidden; otherwise substantive.

```
variable {G : Type*} [Group G]

example (a b : G) :
    (a * b)⁻¹ = b⁻¹ * a⁻¹ := by
  ...
```

> Natural language says "product" without naming the algebraic structure. Formal language cannot use multiplication until the structure is fixed.

Every Subgroup Of A Cyclic Group Is Cyclic

Informal:

> Every subgroup of a cyclic group is cyclic.

Hidden choices

- Ambient group G.
- Multiplicative vs additive notation.
- "Cyclic" as a predicate on groups, subgroups, or modules?
- Subgroup as bundled `Subgroup G`?
- Subgroup cyclicity uses the inherited group structure?

```
variable {G : Type*} [Group G]

-- teaching approximation,
-- not a verified final API claim:
∀ H : Subgroup G,
  IsCyclic G → IsCyclic H
```

> "Subgroup" in natural language refers both to a subset-like object and to a group in its own right. Formalization must choose the bundled representation that exposes the needed structure.

Continuous Functions On $[a, b]$ Attain A Maximum

Informal:

> A continuous function on $[a, b]$ attains a maximum.

Hidden choices

- $a \leq b$ assumed?
- $f : \mathbb{R} \to \mathbb{R}$, or $f : X \to \mathbb{R}$?
- `Continuous f` vs `ContinuousOn f (Set.Icc a b)`?
- "Attains a maximum" means $\exists x \in [a, b], \forall y \in [a, b], f(y) \leq f(x)$?
- Is this really compact-set extreme value?

```
∃ x ∈ Set.Icc a b,
  ∀ y ∈ Set.Icc a b, f y ≤ f x
```

A formal statement here specifies:

- domain set: `Set.Icc a b`;
- codomain order: $\leq$ on $\mathbb{R}$;
- continuity predicate: `ContinuousOn`;
- compactness source: closed bounded interval.

A common theorem name often hides compactness, order, and domain restriction.

$\mathrm{Gal}(E/\mathbb{Q}) \cong Q_8$: One Statement, Several Formal Shapes

Algebra at PhD level. The goal here is **not** the algebra content; observe how many alignment choices a single informal isomorphism forces.

Informal:

Let $\alpha = \sqrt{(2+\sqrt{2})(3+\sqrt{3})}$ and $E = \mathbb{Q}(\alpha)$.
Show $\mathrm{Gal}(E/\mathbb{Q}) \cong Q_8$.

Hidden choices

- α as `Real.sqrt …`;
- E as `IntermediateField.adjoin ℚ {α}`;
- $\mathrm{Gal}(E/\mathbb{Q})$ as `↥E ≃ₐ[ℚ] ↥E`;
- Q_8 as `QuaternionGroup 2`;

```
-- Shape A: explicit MulEquiv (def, not theorem)
def gal_iso_Q8 :
  (↥E ≃ₐ[ℚ] ↥E) ≃* QuaternionGroup 2 := by
  sorry

-- Shape B: Galois-group predicate
theorem q8_is_galois_group
    [MulSemiringAction (QuaternionGroup 2) ↥E] :
    IsGaloisGroup
      (QuaternionGroup 2) ℚ ↥E := by
  sorry

-- Shape C: finite Galois group object
noncomputable def q8_as_finGaloisGroup_iso :
    Efin.finGaloisGroup
      ≅ FiniteGrp.of (QuaternionGroup 2) := by
  sorry
```

Three Shapes, Three API Paths

Shape A – MulEquiv

Direct group isomorphism.

```
(↥E ≃ₐ[ℚ] ↥E) ≃* QuaternionGroup 2
```

Closest to the informal $\cong$. Must use `def` (the type is `Type`, not `Prop`).

Shape B – IsGaloisGroup

Q_8 acts faithfully on E with fixed field $\mathbb{Q}$.

```
IsGaloisGroup (QuaternionGroup 2) ℚ ↥E
```

Mathlib bridges to `MulEquiv` via `IsGaloisGroup.mulEquivAlgEquiv`.

Shape C – FiniteGrp

Package E as `FiniteGaloisIntermediateField`.

```
Efin.finGaloisGroup ≅ FiniteGrp.of (QuaternionGroup 2)
```

"Finite Galois" lives inside the object, not as a separate hypothesis.

Three reasonable formal targets for one informal isomorphism. Choice of shape changes available API and downstream proof routes.

Noetherian Local Ring: Regular Sequences and Systems of Parameters

Commutative algebra at PhD level. The point is the alignment surface, not the algebra.

Informal (Big Cohen–Macaulay):

Let R be Noetherian commutative local with maximal ideal $\mathfrak{m}_R$. There exists an R-module W such that $\mathfrak{m}_R W \neq W$ and every system of parameters for R is a regular sequence on W.

Hidden choices

- $\mathfrak{m}_R W$ as `m_R • (⊤ : Submodule R W)`;
- regular sequence via `IsSMulRegular` on iterated quotients;

```
def IsRegularSeq
    (R : Type*) [CommRing R]
    (M : Type*) [AddCommGroup M] [Module R M] :
    List R → Prop
  | []     ⇒ True
  | r :: t ⇒
    IsSMulRegular M r ∧
    IsRegularSeq R
      (M ⧸ (Ideal.span ({r} : Set R)
              • (⊤ : Submodule R M))) t

def IsSystemOfParameters
    (R : Type*) [CommRing R] [IsLocalRing R]
    {n : ℕ} (s : Fin n → R) : Prop :=
  (Ideal.span (Set.range s)).radical
      = IsLocalRing.maximalIdeal R ∧
  (n : WithBot ℕ∞) = ringKrullDim R

theorem exists_module_sop_regular
    (R : Type*) [CommRing R] [IsLocalRing R]
    [IsNoetherianRing R] :
  ∃ (W : Type*) (_ : AddCommGroup W)
    (_ : Module R W),
    IsLocalRing.maximalIdeal R • (⊤ : Submodule R W)
```

Part IV. Modern Autoformalization Systems

Mechanisms aimed at the alignment problem

What Modern Systems Are Trying To Fix

> The formal statement can compile and still not be the intended theorem.

These papers are not surveyed as independent contributions. Each is presented as a mechanism that addresses one aspect of alignment.

Aspects covered in Part IV:

- library vocabulary selection;
- dependency retrieval;
- local definition synthesis;
- semantic decomposition;
- statement validation;
- statement repair;
- project-level prose / declaration correspondence.

Rethinking: Typecheck Is Too Weak

> *Lean accepts the generated statement*
> *does not imply*
> *the generated statement is the intended theorem.*

Common metrics each have a failure mode:

- typecheck – surface legality;
- BLEU – string similarity;
- LLM judge – surface text;
- definitional equality – too strict.

BEq (Bidirectional Extended Definitional Equivalence): both directions must derive from the other under bounded symbolic / neural transformations.

200 expert-labeled formal pairs:

Metric	P	R	Acc
Typecheck	35.0	100.0	35.0
BLEU	63.0	24.3	68.5
LLM majority	70.6	85.7	82.5
Def. equality	100.0	11.4	69.0
BEq	100.0	72.9	90.5

Paper examples: confusing `Real.pi` with a free variable `pi`; treating a Lean default argument `(f : Polynomial ℤ := X^6 + ...)` as a mathematical equation.

Rethinking: Dependency Retrieval Grounds Vocabulary

> Failure mode: *agnosia of context*. The model can write Lean; it cannot see the surrounding formal world.

Authors construct a Mathlib 4 dependency dataset:

```
243,797 formal objects
139,933 theorems
```

Method: parse dependency graph; informalize in topological order; basic objects first; their NL descriptions feed the next layer.

Retrieval is not plain semantic search. BM25 on "holomorphic / set / constant" favors documents that repeat those words, while the actual dependency may be a basic definition like `Set` whose surface frequency is low.

Reported Recall@5:

```
ProofNet   35.52 %
Con-NF     24.32 %
```

> "The integral of f from a to b" possibly depends on: - `intervalIntegral` - `MeasureTheory.Integral.Bochner` - `BoxIntegral.Riemann` - `IntegrableOn` - `HasDerivAt` / `HasDerivWithinAt` - `ContinuousOn`

Rethinking: Autoformalization Improves When Dependencies Are Present

RAutoformalizer places retrieved dependencies in the autoformalization prompt / training context.

```
ProofNet BEq@8:
  baseline          12.83 %
  RAutoformalizer   18.18 %

Con-NF BEq@8:
  baseline          4.58 %
  RAutoformalizer  16.86 %
```

Oracle-dependency setting is much higher, especially on Con-NF: bottleneck is *which formal objects are in context*, not raw generation skill.

A three-layer check:

Check	Question	Misses
Typecheck	Legal Lean?	Whether intended theorem
BEq	Equivalent to reference?	Whether reference aligns to NL
Dep retrieval	Right formal vocabulary?	Whether deps are enough / noisy

> A practical lesson: prompt should not say "translate this theorem into Lean", but rather "identify the formal objects expressing each concept, then write the statement using them".

Aria: Concept Dependency Graph Before Lean Code

Failure modes Aria targets: - hallucinated Mathlib identifier; - outdated API; - nearby but wrong formal object; - concept missing from Mathlib needs local synthesis; - synthesized def compiles but is wrong.

Graph-of-Thought pipeline:

```
informal research statement
  → concept dependency graph
  → LeanSearch grounding for leaves
  → bottom-up synthesis of
     missing internal concepts
  → compiler-in-the-loop reflection
  → final Lean theorem statement
  → AriaScorer term-level check
```

Statement alignment is not the last line of a theorem header. It is the alignment of an entire concept graph.

Aria: Homological Conjecture Case Study

> Same statement as the Noetherian local ring example earlier: Big Cohen–Macaulay module existence.

Gemini baseline

```
theorem exists_bcm_gemini
    (R : Type*) [CommRing R] [LocalRing R]
    [IsNoetherianRing R] :
    ∃ (W : Type*) (_ : Module R W),
      LocalRing.maximalIdeal R • W ≠ W ∧
      ∀ s, IsSystemOfParameters R s →
        IsRegularSequence W s := by
  sorry
```

Three errors:

- `IsSystemOfParameters` hallucinated;
- `IsRegularSequence` is not the Mathlib API;
- `m_R • W` on a type, skips submodule `⊤`.

Aria after GoT + RAG + synthesis

Same statement as the Noetherian local ring example.

Layer	Gemini	Aria
ideal · module	`m_R • W` (type)	`m_R • ⊤` (submodule)
sys of params	hallucinated	local synth
regular seq	wrong name	grounded API

> Failure happens in two places at once: grounding (wrong / fake names) and synthesis (concept not decomposed). Aria's GoT separates these as graph nodes.

AriaScorer: Term-Level Grounding

> *If a generated local definition compiles, how do we know it is the intended mathematical concept?*

AriaScorer uses `jixia` static analysis: for each term in the Lean statement, retrieve `name, kind, type, value, informal name, informal description` from the informalized Mathlib dataset. The LLM judge compares atomic assumptions and conclusions in that grounded context.

Case studies

- **UFD / IsDomain.** "UFD" implies integral domain; surface checker may flag a "missing" condition already inside the UFD class.
- **Quaternion algebra.** Same surface name, different multiplication rules.
- **QuaternionGroup 1 vs 2.** Index hides the difference between order 4 and order 8.
- **Catenary ring.** Local def silently adds `extends IsNoetherianRing` – semantic drift.

Headline accuracies: ProofNet 68.5 %, FATE-H 71.0 %, FATE-X 44.0 %, homological conjectures 42.9 %.
Ablations: remove RAG $\rightarrow$ conjecture accuracy $0\,\%$; remove GoT $\rightarrow$ 6/14 $\rightarrow$ 1/14.

LoC-Decomp: Formalization Template Exposes Commitments

LoC-Decomp does not ask the model for a one-shot theorem header. It enforces a template:

```
environment imports
auxiliary types
mathematical systems
auxiliary functions
constraints
solution constraints
final problem statement
```

A formalization template is chain-of-thought structure for code.

Figure 2 example:

```
def continuous_at (x₀ : ℝ) (f : ℝ → ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x,
    |x - x₀| < δ → |f x - f x₀| < ε
```

```
System: a, b, sol_sum_a_b
function_constraint:
  f x = if x > 2 then a*x + 3
        else if -2 ≤ x ∧ x ≤ 2 then x - 5
        else 2*x - b
continuous_constraint:
  ∀ x₀, continuous_at x₀ f
solution_constraint:
  sol_sum_a_b = a + b
problem:
  every system satisfying constraints
  has solution = sol_sum_a_b
```

LoC-Decomp: ASCC Back-Translation Is Structured

Automatic Semantic Consistency Check uses divide–conquer–merge:

```
Lean formalization
  → divide into semantic segments
  → translate each segment to NL
  → merge segment translations
  → compare with original problem
  → identify discrepancies
  → classify severity
  → propose rectification
```

Segment-level back-translation, not whole-file.

Two repair channels alternate:

> **Semantic repair** from ASCC discrepancy feedback.

> **Compiler repair** from Lean error messages.

State	Meaning
sem correct, compile fail	math right, API wrong
compile ok, sem wrong	wrong theorem proved
both fail	not yet evaluable
both pass	alignment evidence, not equivalence proof

LoC-Decomp: Evidence And Classroom Mini-Example

ASCC-3-MV vs human labels (MATH-ASCC-Eval-150):

```
precision 0.90
recall    0.82
F1        0.86
```

DeepSeek-V3 + LoC-Decomp template (pass rate):

```
no iter rect.
  MATH-500 63.20  miniF2F 77.25

with iter rect.
  MATH-500 84.00  miniF2F 90.16
```

PutnamBench numbers differ across evaluation variants; safer claim: structured decomposition plus feedback loop consistently improves reported pass rates.

> Mini-example: *If f is continuous on $[a, b]$, then f attains a maximum on $[a, b]$.*

```
object       f : ℝ → ℝ
object       a b : ℝ
domain       K := Set.Icc a b
constraint   h_ab : a ≤ b
constraint   h_cont : ContinuousOn f K
conclusion   ∃ x ∈ K, ∀ y ∈ K, f y ≤ f x
```

Two wrong candidates to discuss:

- `Continuous f` instead of `ContinuousOn f K` – too strong assumption.
- `∃ x, ∀ y, f y ≤ f x` – global max, too strong conclusion.

ReForm: Reflection Loop For Semantic Fidelity

LoC-Decomp uses an external semantic checker. ReForm trains the model to do generation, critique, and revision itself.

```
natural-language problem Q
  → formal statement S₁
  → semantic critique C₁
  → revised statement S₂
  → semantic critique C₂
  → ...
  → final answer
```

History $H_t = \{(S_1, C_1), \ldots, (S_{t-1}, C_{t-1})\}$ feeds each round.

FTC candidate:

```
∫ x in a..b, f x = F b - F a
```

A specific critique would catch:

- missing assumption $F' = f$;
- missing continuity / integrability;
- derivative on interval vs globally;
- oriented interval convention;
- FTC-1 vs FTC-2.

ReForm: PBSO Rewards The Critique, Not Only The Answer

Reward only on final answer ⇒ shallow critique survives. PBSO splits rewards by sequence position:

```
task reward:
  final statement passes Lean
  and is semantically consistent with Q

auxiliary reward:
  intermediate critique faithfully
  diagnoses S_t vs Q
```

Shallow critique (penalize):

"The formal statement uses standard Lean notation. It correctly expresses the equality. The syntax is valid; the statement matches the problem."

Specific critique (reward):

"Q states a conditional: 'if $F' = f$, then …'. S_1 integrates f but never introduces F as an antiderivative. Missing:
(a) HasDerivAt F (f x) x;
(b) integrability of f on $[a, b]$.
Also: HasDerivAt vs HasDerivWithinAt unspecified."

Good critique:

- cites Q's structure;
- names the missing commitment;

ReForm: Evidence And Limits

Semantic consistency (main table):

Model	miniF2F	ProofNet	Putnam	AIME
ReForm-8B	87.7	65.6	57.3	46.7
ReForm-32B	91.4	70.4	62.3	66.7

Average $+22.6$ pp over strongest comparable baseline.

ConsistencyCheck (859 expert items):

> miniF2F human-written formal statements: **16.4 %** contain semantic errors.
> ProofNet human-written formal statements: **38.5 %** contain semantic errors.

Examples:

- ProofNet: $\sqrt{11}$ rewritten as 11 in Lean;
- $\deg \leq 80$ rewritten as $\deg < 80$;
- miniF2F: explicit answer baked into the formal statement when the original problem did not give one.

Roundtrip Verification: No Gold Formal Target

Benchmark setting:

```
informal x
gold formal y_gold
candidate y
compare y with y_gold
```

Real formalization more often:

```
informal x
no gold formal target
candidate y must be audited
```

Roundtrip loop:

```
x       informal statement
y_orig  = formalize(x)
x'      = informalize(y_orig)
y_rt    = formalize(x')
check:
  y_orig ≡ y_rt
```

If y_{orig} and y_{rt} are equivalent, the pipeline at least stabilizes on a formal meaning. If not, semantic drift entered one of three stages.

Roundtrip: Stage Diagnosis And Scoped Repair

```
T1: informal → formal
T2: formal   → informal
T3: informal → formal
```

When $y_{orig} \not\equiv y_{rt}$, a diagnoser inspects $(x, y_{orig}, x', y_{rt})$ and predicts which stage first failed. Repair is scoped:

- repair T1 – y_{orig} changes, regenerate T2, T3;
- repair T2 – x' changes, regenerate T3;
- repair T3 – only y_{rt} changes.

A practical debugging mental model:

> Was the first Lean statement wrong?
> Was the back-translation misleading?
> Was the second formalization unstable?

Localizing by stage parallels LoC-Decomp localizing by segment.

Roundtrip: Evidence, Drift, And Fixed Points

Domains: Texas Transportation Code (150 rules) and Parks & Wildlife Code (77 rules). Models: Claude Opus 4.6, GPT-5.2. Equivalence: Z3.

```
No repair:                  44.7 - 66.2 %
Claude + scoped repair:     85.3 / 85.7
GPT + full repair:          82.7 / 77.9
GPT pipeline + Claude diag: 90.7 / 94.7
```

GPT's sequential diagnosis prompt blames T1 in $\approx 83\,\%/97\,\%$ of cases. A non-sequential prompt drops that to $9\,\%/16\,\%$ – replacing only the diagnoser improves results.

> Roundtrip consistency is evidence, not proof. The loop can stabilize on a wrong fixed point:
>
> `x → y_wrong → x_wrong → y_wrong`

For Lean mathematics, $y_{orig} \equiv y_{rt}$ may need Mathlib theorems, typeclass search, coercion reasoning, or domain lemmas – sometimes itself a new theorem, not an SMT check.

Herald: Mathlib4 To Natural-Language Data

Pipeline:

```
Mathlib4 declarations and proofs
  → Lean-jixia extracts names,
    variables, dependencies,
    proof states, tactic steps,
    surrounding context
  → dependency-aware informalization
  → tactic / informal augmentation
  → Herald dataset
  → Herald Translator
```

Reported scale: 580k valid statements, 44k NL-FL theorem pairs, 45k NL-FL proof pairs.

`Set.Ico` walkthrough – raw Lean:

```
theorem foo (a b c : ℝ) :
    c ∈ Set.Ico a b ↔ a ≤ c ∧ c < b
```

Naive output:

```
For real numbers a, b, c, c belongs
to the set Ico from a to b iff
a ≤ c and c < b.
```

Herald with jixia metadata:

```
For real numbers a, b, c, the element
c lies in the left-closed right-open
interval [a, b) iff a ≤ c < b.
```

jixia surfaces `Set.Ico := { x | a ≤ x ∧ x < b }` and its informal name; the LLM is the same, the context is not.

LeanSearch As Downstream Infrastructure

Herald-style data feeds a search service:

```
NL query
  → embedding model trained
    on (informal, formal) pairs
  → nearest-neighbor over
    informalized Mathlib
  → formal declarations
```

LeanSearch is the deployed form of this. It already appeared as Aria's grounding step earlier.

The FL → NL chain:

```
Herald
  dataset + translator

LeanSearch
  informalized Mathlib as a service

Aria / Rethinking
  LeanSearch-style retrieval
  inside NL → FL pipelines
```

FL → NL is not the symmetric counterpart of NL → FL. It is upstream infrastructure: without informalized Mathlib at scale, retrieval and grounding cannot happen.

Herald Translator: Validation And Failure Modes

```
miniF2F-test                  96.7 %
internal graduate textbook    23.5 %
College CoT                   16.0 %
```

> Contest-style: very strong.
> Graduate / modern math: much harder.

Validation pipeline:

```
generated formal statement
  → Lean compiler check
  → back-translation to NL
  → NLI check against original
```

Failure modes with examples:

- **Subsequence.** Index map l must `Tendsto l atTop atTop`.
- **Closed subspace.** Missing "closed" $\longrightarrow$ NLI rejects; extra Y, f also rejected.
- **False negative.** Correct formal statement; back-translation flips implication direction.

Eigenvector Quantifier Order

Informal: *Every vector is an eigenvector of* T. Ambiguous in NL; Lean must fix one.

Shape A – $\forall v, \exists k$:

```
∀ v : V, ∃ k : F, T v = k • v
```

"Each v has its own scalar k_v." Proof obligation: build a function $v \mapsto k_v$.

Shape B – $\exists k, \forall v$:

```
∃ k : F, ∀ v : V, T v = k • v
```

"A single scalar k works for every v." Proof obligation: produce a v-independent constant.

Translator must commit to A or B. Back-translation / NLI catches silent quantifier flip during round-trip.

Herald Case: Four Lemma, Mono Half

```
theorem
  CategoryTheory.Abelian.mono_of_epi_of_mono_of_mono
    {C : Type u_1} [Category.{v_1, u_1} C]
    [Abelian C]
    {R₁ R₂ : ComposableArrows C 3}
    (φ : R₁ ⟶ R₂)
    (hR₁ : R₁.Exact) (hR₂ : R₂.Exact)
    (h₀ : Epi (ComposableArrows.app' φ 0 ...))
    (h₁ : Mono (ComposableArrows.app' φ 1 ...))
    (h₃ : Mono (ComposableArrows.app' φ 3 ...)) :
    Mono (ComposableArrows.app' φ 2 ...)
```

The `...` are Lean-generated index proofs, not mathematical content.

A faithful informalization should restore the mathematical surface:

> **Four Lemma (monomorphism half).** Let C be abelian. Suppose a morphism between two exact composable chains of length 3, with the squares commuting. If φ_0 is epi, φ_1, φ_3 are mono, then φ_2 is mono.

Do not unfold Lean implementation details (e.g. `ComposableArrows.app'` index proofs) into the informal version.

LeanArchitect: Blueprint Drift Is Alignment Drift

Big Lean projects keep a LaTeX blueprint: informal plan, proof sketches, dependency graph, Lean links, progress markers. Two truth sources:

```
LaTeX blueprint:
  informal theorem,
  proof sketch,
  dependency labels,
  progress markers

Lean files:
  declarations, proofs,
  imports, constants used
```

> Drift is dangerous when proof agents treat the blueprint as a task graph: - working on a stale node; - assuming a missing dependency exists; - missing a dependency edge; - trusting a wrong `\leanok`; - formalizing a theorem whose prose has changed.

Alignment is no longer one theorem statement; it spans prose, declarations, dependencies, and proof status.

LeanArchitect: `@[blueprint]` Moves Metadata Into Lean

```
@[blueprint "thm:add-comm"
  (statement := /-- Addition in $N$
                   is commutative. -/)]
theorem MyNat.add_comm (a b : MyNat) :
    a + b = b + a := by
  /-- By induction and then
      \cref{lem:zero-add, lem:succ-add}. -/
  ...
```

Lean is the source. LaTeX is generated.

Extracted automatically:

```
Lean name
LaTeX label
NL statement
NL proof sketch
deps from constants in type/value
proof status from sorry-free check
```

Generated LaTeX fragments:

```
\lean{MyNat.add_comm}
\leanok
\uses{def:nat}
\mathlibok
```

Auto-sync $\neq$ semantic equivalence. Metadata stays current; whether NL and Lean express the same claim still needs audit.

LeanArchitect Case: Multivariate Taylor And `derivWithin`

Multivariate Taylor theorem in integral form.
Workflow:

```
GPT-5 Pro
  drafts NL proof + Lean module
  with @[blueprint] nodes and sorrys

blueprint graph
  visualizes deps and progress

Aristotle
  fills sorry nodes

human / GPT
  adjusts failures
```

After refinement, Aristotle proved all but three nodes. The remaining failures exposed a statement-level alignment bug:

> Initial formalization used `deriv` for a function on an interval $[a, b]$. Correct formal object: `derivWithin`.

Isomorphic to the FTC running example: derivative on a restricted domain is not the ambient derivative.

Part IV Synthesis

Better metric

Rethinking: typecheck → BEq.

Aria: term-level grounding.

ReForm: ConsistencyCheck audit of "gold" targets.

Library grounding

Rethinking: dependency retrieval.

Aria: concept dependency graph + bottom-up synthesis.

Herald + LeanSearch: the data and service that make grounding possible.

Reference audit

ReForm: reference statements have errors (16.4 % / 38.5 %).

Roundtrip: evidence without a gold target.

LeanArchitect: project-scale prose / declaration sync.

> Three layers to remember: semantic-level check beyond typecheck; library grounding via retrieval and synthesis; gold-target audit and project-scale alignment.

Part V. In-Class Activity

Formalization decision record

Students Do Not Prove; They Specify

Each group fills a formalization decision record:

```
Informal statement:

Objects:

Types:

Structures / typeclasses:

Definitions from library:

Quantifiers:

Assumptions:

Conclusion:

Possible edge cases:

Back-translation:
```

> Output is not a proof. Output is a justified formal statement.

Suggested Group Prompts

Prompt A. The inverse of a product is the product of inverses in reverse order.

Prompt B. A continuous function on $[a, b]$ attains a maximum.

Prompt C. Every subgroup of a cyclic group is cyclic.

Prompt D. The first n odd numbers sum to n^2.

Prompt E. The indefinite integral of f is F.

Deliberately ambiguous: forces choice among antiderivative relation, family of primitives, or base-point integral.

Alignment Review Questions

After each presentation, the class asks:

1. If this Lean statement is true, does the original informal claim follow?
2. If the original informal claim is true, does this Lean statement follow?
3. Did the group sneak in an assumption?
4. Did the group drop a needed assumption?
5. Did the group swap the object?
6. Does the back-translation still sound like the original theorem?

> This is the classroom version of semantic validation.

Part VI. Statement Before Proof

Bridge to Session 5

What Session 4 Leaves Behind For Session 5

Session 4 ends at:

If the statement is wrong, a correct proof only